

```
In [ ]: import marimo as mo
```

Knowledge Distillation: ResNet50 → ResNet18 on CIFAR-10

Basic Setup

```
In [ ]: import os
import time
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import random_split, DataLoader
from torch.optim.lr_scheduler import ReduceLROnPlateau
from torchvision import models
import torchvision.transforms as transforms
from torchvision.datasets import CIFAR10

print(f"PyTorch version: {torch.__version__}")
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Device used: {device.type}")
```

PyTorch version: 2.12.1

Device used: cuda

```
In [ ]: import numpy as np
import random

def set_seed(seed: int = 42):
    random.seed(seed)
    np.random.seed(seed)
    os.environ["PYTHONHASHSEED"] = str(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)  # For multi-GPU

    # Configure CUDA backends
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

    # Force use of deterministic algorithms (PyTorch 1.7+)
    torch.use_deterministic_algorithms(True)

set_seed(42)
```

Load Dataset

```
In [ ]: transform = transforms.Compose(
    [
```

```

        transforms.ToTensor(),
        transforms.Normalize(
            mean=[0.4914, 0.4822, 0.4465], # CIFAR-10 means
            std=[0.2023, 0.1994, 0.2010],
        ),
    ],
)

# Load CIFAR-10 train set
full_trainset = CIFAR10(root="./data", train=True, download=True, transform=
full_trainset

```

Dataset CIFAR10

Number of datapoints: 50000

Root location: ./data

Split: Train

StandardTransform

Transform: Compose(

ToTensor()

Normalize(mean=[0.4914, 0.4822, 0.4465], std=[0.2023, 0.1994, 0.201])

)

```

In [ ]: train_size = int(0.9 * len(full_trainset))
        val_size = len(full_trainset) - train_size

        # Perform split
        train_subset, val_subset = random_split(full_trainset, [train_size, val_size])
        print(f"Train samples: {train_size}")
        print(f"Validation samples: {val_size}")

```

Train samples: 45000

Validation samples: 5000

```

In [ ]: train_loader = DataLoader(train_subset, batch_size=128, shuffle=True)
        val_loader = DataLoader(val_subset, batch_size=128, shuffle=False)

```

```

In [ ]: test_set = CIFAR10(root="./data", train=False, download=True, transform=train_loader.transform)
        test_loader = DataLoader(test_set, batch_size=128, shuffle=False)
        print(f"Test samples: {len(test_set)}")

```

Test samples: 10000

Setup Models

```

In [ ]: class FeatureProjector(nn.Module):
        """
        Feature projector to match student -> teacher feature shapes
        """

        def __init__(self, in_channels, out_channels):
            super(FeatureProjector, self).__init__()

```

```

        # define a 1x1 convolutional layer to project feature maps
        self.proj = nn.Conv2d(in_channels, out_channels, kernel_size=1)

    def forward(self, x, target_shape):
        # check if the spatial dimensions of the input match the target shape
        if x.shape[2:] != target_shape[2:]:
            # adjust spatial dimensions using adaptive average mapping
            x = F.adaptive_avg_pool2d(x, output_size=target_shape[2:])

        return self.proj(x)

```

```

In [ ]: class StudentWrapper(nn.Module):
    """
    Wrapper for the student model with projection layers
    """

    def __init__(self, student_model, proj_layers):
        super(StudentWrapper, self).__init__()

        self.model = student_model

        # store projection layers for feature alignment
        self.projections = nn.ModuleList(proj_layers)

    def forward(self, x):
        # collect intermediate features from ResNet blocks
        features = []
        x = self.model.conv1(x)
        x = self.model.bn1(x)
        x = self.model.relu(x)
        x = self.model.maxpool(x)

        for i, block in enumerate(
            [self.model.layer1, self.model.layer2, self.model.layer3, self.model.layer4]):
            x = block(x)

            # append features from each block
            features.append(x)

        # pool the final feature map and compute logits
        pooled = F.adaptive_avg_pool2d(x, (1, 1))
        flat = torch.flatten(pooled, 1)
        logits = self.model.fc(flat)

        return logits, features

    def project_features(self, features, target_shapes):
        """
        Project student features to match the shapes of teacher features.
        """
        return [
            proj(s_feat, t_shape)
            for s_feat, t_shape, proj in zip(features, target_shapes, self.projections)
        ]

```

```
In [ ]: def setup_models(device):
        """
        Setup teacher and student wrapper
        """

        # teacher: ResNet50 pretrained on ImageNet, re-headed for CIFAR-10
        teacher = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)
        teacher.fc = nn.Linear(2048, 10)
        teacher = teacher.to(device)

        # student: ResNet18 without pretrained weights
        student = models.resnet18(weights=None)
        student.fc = nn.Linear(512, 10)
        student = student.to(device)

        # define the intermediate feature channels for both teacher and student
        teacher_channels = [256, 512, 1024, 2048]
        student_channels = [64, 128, 256, 512]

        # create projection layers to align teacher's feature maps with student's
        proj_layers = [
            FeatureProjector(in_c, out_c).to(device)
            for in_c, out_c in zip(student_channels, teacher_channels)
        ]

        # wrap the student model with the projection layers
        student_wrapper = StudentWrapper(
            student_model=student,
            proj_layers=proj_layers,
        )

        return teacher, student_wrapper
```

```
In [ ]: def extract_teacher_features(model, x, layers=[1, 2, 3, 4]):
        """
        Extract teacher logits and intermediate features
        """

        # collect intermediate features from ResNet blocks
        features = []
        x = model.conv1(x)
        x = model.bn1(x)
        x = model.relu(x)
        x = model.maxpool(x)
        for i, block in enumerate([model.layer1, model.layer2, model.layer3, model.layer4]):
            x = block(x)
            if (i + 1) in layers:
                features.append(x)

        # pool the final feature map and compute logits
        pooled = F.adaptive_avg_pool2d(x, (1, 1)) # [B, C, 1, 1]
        flat = torch.flatten(pooled, 1) # [B, C]
        logits = model.fc(flat) # [B, 10]
        return logits, features
```

Evaluation Functions for Size, Latency and Accuracy

```
In [ ]: def count_params(model):  
        """  
        Function to count trainable parameters  
        """  
  
        return sum(p.numel() for p in model.parameters() if p.requires_grad)
```

```
In [ ]: def measure_latency(model, input_size=(1, 3, 32, 32), device="cuda", repetitions=10):  
        """  
        Function to measure average inference latency over multiple runs  
        """  
  
        model.eval()  
        inputs = torch.randn(input_size).to(device)  
        with torch.no_grad():  
            # Warm-up  
            for _ in range(10):  
                _ = model(inputs)  
  
            # Measure  
            durations = []  
            for _ in range(repetitions):  
                start = time.time()  
                _ = model(inputs)  
                end = time.time()  
                durations.append(end - start)  
        return (sum(durations) / repetitions) * 1000
```

```
In [ ]: def evaluate_accuracy(model, dataloader):  
        """  
        Evaluate accuracy given model and loader  
        """  
  
        model.eval()  
        model.to(device)  
        correct, total = 0, 0  
        with torch.no_grad():  
            for inputs, labels in dataloader:  
                inputs, labels = inputs.to(device), labels.to(device)  
                outputs = model(inputs)  
                preds = outputs.argmax(dim=1)  
                correct += (preds == labels).sum().item()  
                total += labels.size(0)  
        accuracy = correct / total  
        return accuracy
```

Fine-tuning the Teacher

Note: This fine-tuning is required only in this example since the ResNet50 network was pretrained on ImageNet1K and we need to replace its output layer to match to CIFAR-10.

```
In [ ]: import subprocess

# Run a simple command
result = subprocess.run(["mkdir", "models"], capture_output=True, text=True)

# Print the output
print(result.stdout)
```

```
In [ ]: @mo.persistent_cache
def train_teacher(
    teacher: torch.nn.Module,
    train_loader,
    val_loader,
    epochs,
    tag,
    lr=1e-3,
    model_file_name="models/teacher_model.pt",
):
    """
    Trains a model with Adam and cross-entropy loss.
    Loads from save_path if it exists.
    """

    full_save_path = f"./models/{model_file_name}"
    if os.path.exists(full_save_path):
        print(f"Teacher model is already trained. Loading from {full_save_path}")
        teacher.load_state_dict(torch.load(full_save_path), strict=True)
        return teacher

    # No saved model found. Training from given model state

    optimizer = torch.optim.Adam(teacher.parameters(), lr=lr)
    teacher.train()

    best_validation_accuracy = 0.0

    for epoch in range(epochs):
        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)

            optimizer.zero_grad()

            logits, _ = extract_teacher_features(teacher, inputs)
            loss = F.cross_entropy(logits, labels)
            loss.backward()
            optimizer.step()

        validation_accuracy = evaluate_accuracy(teacher, val_loader)
        print(
            f"({tag})\tEpoch {epoch + 1}: loss={loss.item():.4f}, Accuracy ("
```

```

    )

    if validation_accuracy > best_validation_accuracy:
        best_validation_accuracy = validation_accuracy
        torch.save(teacher.state_dict(), full_save_path)
        print("New best teacher model is saved.")

    # load best checkpoint
    teacher.load_state_dict(torch.load(full_save_path), strict=True)
    return teacher

set_seed(42)
teacher, student_wrapper = setup_models(device)
teacher = train_teacher(
    teacher,
    train_loader,
    val_loader,
    epochs=25,
    tag="Fine-tuning teacher",
    model_file_name="tuned_pretrained_resnet50_on_CIFAR10.pt",
)

```

Teacher model is already trained. Loading from ./models/tuned_pretrained_resnet50_on_CIFAR10.pt

Training the Student via Distillation

The distillation loss is calculated with KL divergence on the student logits and teacher logits.

```

In [ ]: def distillation_loss(teacher_logits, student_logits, targets, T=5.0, alpha=
        """
        Combine soft and hard targets using KL divergence and cross-entropy
        T = temperature, alpha = weighting between soft and hard losses
        """

        # soft target loss (teacher softmax vs student softmax)
        soft_targets = F.kl_div(
            F.log_softmax(student_logits / T, dim=1),
            F.softmax(teacher_logits / T, dim=1),
            reduction="batchmean",
        ) * (T * T)

        # hard label loss
        hard_loss = F.cross_entropy(student_logits, targets)
        return alpha * soft_targets + (1 - alpha) * hard_loss

```

```

In [ ]: def student_training_step(
    inputs,
    labels,
    teacher,
    student_wrapper,
    optimizer,
    device,

```

```

):
    """
    Perform a single training step for the student model using knowledge distillation
    """

    inputs, labels = inputs.to(device), labels.to(device)
    teacher.eval()
    optimizer.zero_grad()

    # extract teacher logits and intermediate features
    with torch.no_grad():
        teacher_logits, teacher_features = extract_teacher_features(teacher, inputs)

    # extract student logits and intermediate features
    student_logits, student_features = student_wrapper(inputs)
    projected_features = student_wrapper.project_features(
        student_features,
        [t.shape for t in teacher_features],
    )

    # calculate loss from features difference
    feature_loss = sum(
        F.mse_loss(p, t.detach()) for p, t in zip(projected_features, teacher_features)
    )

    # calculate loss from output distribution, and include feature loss
    loss = distillation_loss(student_logits, teacher_logits, labels) + 0.1 * feature_loss

    # optimize with loss
    loss.backward()
    optimizer.step()

    return loss.item()

```

```

In [ ]: @mo.persistent_cache
def train_student(
    teacher,
    student_wrapper,
    train_loader,
    val_loader,
    epochs,
    model_file_name="student_distilled.pt",
):
    """
    Trains a student model using knowledge distillation from a teacher model
    """

    full_save_path = f"./models/{model_file_name}"
    if os.path.exists(full_save_path):
        print(f"Student model is already trained. Loading from {full_save_path}")
        student_wrapper.load_state_dict(torch.load(full_save_path), strict=True)
        return student_wrapper.model

    # setup optimizer
    optimizer = torch.optim.Adam(student_wrapper.parameters(), lr=1e-3)

```

```

# train the student using the teacher's output as soft targets
teacher.eval()

best_validation_accuracy = 0.0

# reduce LR if validation loss doesn't improve for 3 epochs
scheduler = ReduceLROnPlateau(optimizer, mode="min", factor=0.5, patience=3)

for epoch in range(epochs):
    student_wrapper.train()
    running_loss = 0.0
    for inputs, labels in train_loader:
        loss = student_training_step(
            inputs,
            labels,
            teacher,
            student_wrapper,
            optimizer,
            device,
        )
        running_loss += loss

    validation_accuracy = evaluate_accuracy(student_wrapper.model, val_loader)
    print(
        f"[(Training student)\tEpoch {epoch + 1}] Loss = {running_loss / len(train_loader)} "
    )
    scheduler.step(loss)

# save best checkpoint
if validation_accuracy > best_validation_accuracy:
    best_validation_accuracy = validation_accuracy
    torch.save(student_wrapper.state_dict(), full_save_path)
    print("New best student model is saved.")

# load best checkpoint
student_wrapper.load_state_dict(torch.load(full_save_path), strict=True)
return student_wrapper.model

# trigger student training
set_seed(42)
student = train_student(teacher, student_wrapper, train_loader, val_loader,

```

Student model is already trained. Loading from ./models/student_distilled.pt

Model Comparison Code

Finally we check the size of the teacher and student models, their latency and accuracy on test set.

```

In [ ]: teacher_params = count_params(teacher)
        student_params = count_params(student)

teacher_latency = measure_latency(teacher, device=device)
student_latency = measure_latency(student, device=device)

```

```

teacher_acc = evaluate_accuracy(teacher, test_loader)
student_acc = evaluate_accuracy(student, test_loader)

print(f"Teacher Params: {teacher_params / 1e6:.2f}M")
print(f"Student Params: {student_params / 1e6:.2f}M")
print(f"Teacher Latency: {teacher_latency:.2f} ms")
print(f"Student Latency: {student_latency:.2f} ms")
print(f"Teacher Test Accuracy: {teacher_acc * 100:.2f}%")
print(f"Student Test Accuracy: {student_acc * 100:.2f}%")

```

```

Teacher Params: 23.53M
Student Params: 11.18M
Teacher Latency: 8.67 ms
Student Latency: 5.06 ms
Teacher Test Accuracy: 80.81%
Student Test Accuracy: 75.96%

```

Training a baseline student (ResNet18 from scratch)

Although the student model is half the size and 40% the latency of the teacher model, its accuracy dropped from 85.10% to 79.24%. To determine whether this is still a better approach compared to training the student model directly on the data, we trained another student model for the same number of epochs without using knowledge distillation. We refer to this model as the “baseline student”.

```

In [ ]: baseline_student = models.resnet18(weights=None)
baseline_student.fc = nn.Linear(512, 10).to(device)
baseline_student = baseline_student.to(device)

# Train the baseline student on CIFAR-10
baseline_student = train_teacher(
    baseline_student,
    train_loader,
    val_loader,
    epochs=25,
    tag="baseline-student",
    model_file_name="baseline_student.pt",
)

# Evaluate baseline student
baseline_student_acc = evaluate_accuracy(baseline_student, test_loader)
print(f"\nBaseline Student Test Accuracy: {baseline_student_acc * 100:.2f}%")

```

Teacher model is already trained. Loading from ./models/baseline_student.pt

```
Baseline Student Test Accuracy: 71.88%
```